11. Gradient Boosting

Turning Weak Learners into Strong Performers by Learning from Mistakes

Previous Section:

10. Random Forest

Contents:

This section shifts our focus on Boosting, and the best of them all, **Gradient Boosting**.

Up until now, we've seen ensemble methods like Random Forest, where multiple models work together to improve performance. Gradient Boosting also belongs to this idea of **ensemble learning**, but it follows a very different philosophy.

To understand that difference, let's step into a simple classroom.

A Classroom Analogy: Imagine a class of 15 students. The teacher gives them a task: "Find two **prime numbers whose sum is NOT prime".**

Now, there are two ways this class could approach the problem.

- **Bagging (like Random Forest):** All 15 students work **independently**.

    - Each student gives an answer
    - Then they vote
    - The final answer is based on **majority decision**

→ The teacher only reacts to the **final combined answer**, not the individual mistakes.

- **Boosting (what we care about here)**
    - Students work **sequentially**, one after another.
        - **Student 1:** "$2 + 3 = 5$" → Teacher: *Wrong. The sum is prime.*
        - **Student 2 (learns from mistake):** "Okay, sum must not be prime... $2 + 6 = 8$" → Teacher: *Wrong.* 6 *is not a prime.*
        - **Student 3 (learns from both mistakes):** "Both numbers must be prime AND sum must not be prime... $2 + 7 = 9$" → Teacher: *Correct.*

→ At this point, the answer is already found.

Now, of course, real Machine Learning doesn't stop at the 3rd "student" like this example. But the idea is very important:

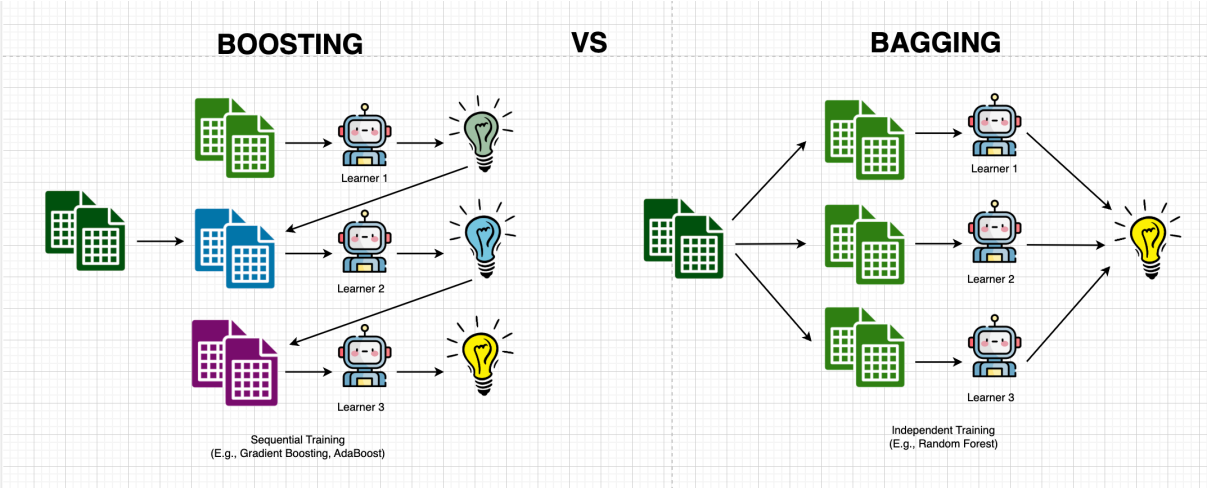
> Each new learner improves by focusing on the **mistakes of the previous ones**

What Makes Boosting Different? → The key idea is simple but powerful:

- The **first model** starts with a very rough understanding (like guessing based on common patterns)
- The **next model** looks at what went wrong and tries to fix it
- The process continues, each model correcting the previous ones

Instead of everyone working separately, learners **build on top of each other**.

Bagging vs Boosting — The Real Difference



- **Bagging (e.g., Random Forest):**
  - Models are trained **independently**
  - Uses **different subsets of data**
  - Combines results at the end (voting or averaging)
- **Boosting (e.g., Gradient Boosting, AdaBoost):**
  - Models are trained **sequentially**
  - Each model focuses on **previous mistakes**
  - The learning process is **adaptive**

Both methods involve multiple learners, and in both cases, the data each model sees can be slightly different. But here's the key distinction:

- In **Bagging**, the difference comes from **different subsets of data**
- In **Boosting**, the difference comes from **how the data is weighted or interpreted based on errors**

In other words: Boosting doesn't just change the data — it changes **what the model pays attention to.**

You might wonder: In Boosting, should we pick subsets of data for each model like we did in Random Forest? Well technically, we *could*. But that's not really how Boosting shines.
In fact, doing so usually ends up being more of a distraction than a help. The magic of Boosting isn't in random subsets—it's in *paying attention to mistakes*, learning from them, and letting each learner

build on what came before. That's what makes it sequential, focused, and powerful.

With that settled, let's step into the world of Gradient Boosting and see exactly how this process turns weak learners into something much stronger.

1. Gradient Boosting – Learning from Mistakes

Now that we've explored Bagging and general Boosting, it's time to meet one of the stars of the ensemble world: **Gradient Boosting**.

Don't worry—it sounds fancy, but it's really just a methodical way of turning a series of "okay" learners into one **super-learner** by paying careful attention to mistakes.

Think of it like a team of robots:

- The first robot tries the task and messes up.
- The second robot studies exactly what the first got wrong and tries to correct it.
- The third robot learns from both the first and second, making fewer errors.
- Repeat this enough times, and you end up with a robot that handles the task extremely well.

Gradient Boosting works the same way. Each new model focuses on the errors left behind by the previous ones, gradually improving the overall performance.

Typically, Gradient Boosting starts with a **base learner**—almost always a decision tree.

At each step, the algorithm looks at the **gradient of the loss function**—that is, how far off its predictions were.

- For regression tasks, this could be **Mean Squared Error (MSE)**,

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- For classification, it could be **Cross-Entropy**.

$$\text{Cross-Entropy} = -[y \log(p) + (1 - y) \log(1 - p)]$$

Mathematically, the process is just keeping track of how wrong each prediction is, and nudging the next model in the direction that reduces that error.

And then, we repeat. Each new model doesn't start from scratch; it builds on the previous mistakes. Slowly, carefully, iteratively, Gradient Boosting transforms weak learners into a strong, highly accurate predictive model. It's widely used in real-world applications because of this relentless focus on minimizing errors and refining predictions.

So in short:

> **Gradient Boosting is Boosting, but smarter**. It's not just about having many models—it's about having models that **learn from each other's mistakes**.

Next, let's move on to the workflow of Gradient Boosting. Think of it like this:

- The first model we train is super simple. We call it **Tree 0**, or sometimes just the **stump tree**. Its job? To make a baseline prediction.
    - If it's a **classification problem**, it predicts the most common label's probability.
    - If it's a **regression problem**, it just predicts the mean value.

It's not magical. It's humble. But here's the important part. We **calculate its mistakes**. These mistakes are called **residuals**, and they're literally the difference between the truth and our model's guess:

$$r_i = y_i - \hat{y}_i$$

Where y_i is the true value, and $\hat{y}_i$ is what Tree 0 guessed.

Now comes the clever bit. We train the **next tree**, Tree 1, not on the original labels, but on these residuals. In other words, this tree learns **where the first model went wrong**.

- This tree is a **regression tree**, because residuals are continuous—even if your original problem is classification.
- Its goal is simple: predict the error, so we know **how to correct our first guess**.

Once Tree 1 is trained, it gives us predicted residuals. We then **update our prediction** like this:

$$\hat{y}_i^{(new)} = \hat{y}_i^{(old)} + \eta \cdot r_i^{(pred)}$$

Here, η is the **learning rate**, a tiny nudge that decides how much we trust this new tree's correction. Tree 1's predictions, $r_i^{(pred)}$, are basically its advice:

> *"Hey, your last guess was off by this much, try adding this."*

And then we repeat. Each new tree looks at the **remaining errors**, tries to fix them, and nudges the overall prediction closer to reality. Step by step, mistake by mistake, the ensemble **learns from itself**, gradually turning a weak, simple model into something strong and reliable.

It's like a team of observers—each watching the previous one, pointing out errors, and suggesting improvements—until finally, they all agree on the best answer.

2. Gradient Boosting – Training Process

My favorite part is always demonstrating an algorithm outside of its theory, because this is where things finally feel real. So let's walk through Gradient Boosting step by step, using this familial dataset, and actually see how the model *learns from its mistakes*.

Age	Income	Student	Credit_Rating	Buy_XBOX
≤30	high	no	Fair	no
≤30	high	no	Excellent	no
31…40	high	no	Fair	yes
>40	medium	no	Fair	yes
>40	low	yes	Fair	yes
>40	low	yes	Excellent	no
31…40	low	yes	Excellent	yes
≤30	medium	no	Fair	no
≤30	low	yes	Fair	yes
>40	medium	yes	Fair	yes

Age	Income	Student	Credit_Rating	Buy_XBOX
≤30	medium	yes	Excellent	yes
31…40	medium	no	Excellent	yes
31…40	high	yes	Fair	yes
>40	medium	no	Excellent	no

- **Step 1: The First Learner — A Humble Beginning**

We begin with something intentionally simple: a **Decision Tree stump**.

This tree doesn't try to be smart. It doesn't split the data. It just looks at the target column (**Buy_XBOX**) and predicts the **most common class** for every single row.

So first, we count:

- $Yes \rightarrow$ (count how many)
- $No \rightarrow$ (count how many)

Whichever appears more becomes our **baseline prediction**.

```
import pandas as pd

data = {'Age': ['≤30', '≤30', '31…40', '>40', '>40', '>40',
'31…40', '≤30', '≤30', '>40', '≤30', '31…40', '31…40', '>4
0'],
        'Income': ['high', 'high', 'high', 'medium', 'low',
'low', 'low', 'medium', 'low', 'medium', 'medium', 'mediu
m', 'high', 'medium'],
        'Student': ['no', 'no', 'no', 'no', 'yes', 'yes',
'yes', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'no'],
        'Credit_Rating': ['Fair', 'Excellent', 'Fair', 'Fai
r', 'Fair', 'Excellent', 'Excellent', 'Fair', 'Fair', 'Fai
r', 'Excellent', 'Excellent', 'Fair', 'Excellent'],
        'Buy_XBOX': ['no', 'no', 'yes', 'yes', 'yes', 'no',
'yes', 'no', 'yes', 'yes', 'yes', 'yes', 'yes', 'no']}

df = pd.DataFrame(data)

# We use Label Encoding for demonstration only
for i in df.columns:
  unique_columns = df[i].unique()
  df[i] = df[i].map({unique_columns[value]: value for value
in range(len(unique_columns))})

# As the first prediction for GB, use probability of the co
mmon class
df['Predicted y_0'] = max(df['Buy_XBOX'].value_counts(norma
lize=True).unique())
print(df[['Buy_XBOX', 'Predicted y_0']])
```

```
   Buy_XBOX  Predicted y_0
0         0      0.642857
1         0      0.642857
2         1      0.642857
3         1      0.642857
4         1      0.642857
5         0      0.642857
6         1      0.642857
7         0      0.642857
```

```
8        1              0.642857
9        1              0.642857
10       1              0.642857
11       1              0.642857
12       1              0.642857
13       0              0.642857
```

So now, every row gets the same prediction. This is what we call: $\hat{y}_0$

It's not accurate—but that's the point. Gradient Boosting doesn't start strong. It starts **honest**.

And now, we let the model face its mistakes.

- **Step 2: Compute the Residuals — Measuring the Mistakes**

Now we ask a very simple question: *How wrong was the model?*

We calculate the **residuals**:

$$r_i = y_i - \hat{y}_0$$

```
df['Residual r_0'] = df['Buy_XBOX'] - df['Predicted y_0']
print(df[['Buy_XBOX', 'Predicted y_0', 'Residual r_0']])
```

```
Buy_XBOX  Predicted y_0  Residual r_0
0        0     0.642857    -0.642857
1        0     0.642857    -0.642857
2        1     0.642857     0.357143
3        1     0.642857     0.357143
4        1     0.642857     0.357143
5        0     0.642857    -0.642857
6        1     0.642857     0.357143
7        0     0.642857    -0.642857
8        1     0.642857     0.357143
9        1     0.642857     0.357143
10       1     0.642857     0.357143
11       1     0.642857     0.357143
12       1     0.642857     0.357143
13       0     0.642857    -0.642857
```

These residuals are no longer just "yes" or "no"—they become **numerical signals of error**.

This is important. Because from this point on, we are no longer predicting labels. We are predicting **mistakes**.

Step 3: The Second Learner — Learning from Errors

Now comes the shift. We train a new model—but not on the original labels. Instead, we train it on the **residuals**.

- Since the target is now the residuals → The model is a **Regression Tree**

This tree tries to answer: *Where did the first model fail, and by how much?*

After training, it produces: $\hat{r}_0$. These are the **predicted residuals**—its attempt to approximate the error.

Now we update our prediction:

$$\hat{y}_1 = \hat{y}_0 + \eta \cdot \hat{r}_0$$

- η is the **learning rate** (a small value like 0.1)
- It controls how much we trust this correction

You *can* even simulate variability here—for learning purposes—by introducing randomness (e.g., using **NumPy**).

```
import numpy as np

# Generate random prediction of the residual
np.random.seed(42)
df['Predict Residual r_0'] = np.random.rand(len(df))
# Update the prediction
learning_rate = 0.1
df['Predicted y_1'] = df['Predicted y_0'] + learning_rate *
df['Predict Residual r_0']
print(df[['Predicted y_0', 'Residual r_0', 'Predict Residua
l r_0', 'Predicted y_1']])
```

	Predicted y_0	Residual r_0	Predict Residual r_0	Predicted y_1
0	0.642857	-0.642857	0.374540	0.680311
1	0.642857	-0.642857	0.950714	0.737929
2	0.642857	0.357143	0.731994	0.716057
3	0.642857	0.357143	0.598658	0.702723
4	0.642857	0.357143	0.156019	0.658459
5	0.642857	-0.642857	0.155995	0.658457
6	0.642857	0.357143	0.058084	0.648666
7	0.642857	-0.642857	0.866176	0.729475
8	0.642857	0.357143	0.601115	0.702969
9	0.642857	0.357143	0.708073	0.713664
10	0.642857	0.357143	0.020584	0.644916
11	0.642857	0.357143	0.969910	0.739848
12	0.642857	0.357143	0.832443	0.726101
13	0.642857	-0.642857	0.212339	0.664091

Think of it like this:

> The first model made a rough guess. The second model whispers: *"Adjust it... just a little."*

- **Step 4: Repeat — Refining the Mistakes**

We don't stop here. Now we:

1. Compute **new residuals** using $\hat{y}_1$
2. Train another tree on these new residuals
3. Update again

$$\hat{y}_2 = \hat{y}_1 + \eta \cdot \hat{r}_1$$

```
df['Residual r_1'] = df['Predicted y_1'] - df['Buy_XBOX']
print(df[['Buy_XBOX', 'Predicted y_1', 'Residual r_1']])
```

	Buy_XBOX	Predicted y_1	Residual r_1
0	0	0.680311	0.680311
1	0	0.737929	0.737929
2	1	0.716057	-0.283943
3	1	0.702723	-0.297277
4	1	0.658459	-0.341541
5	0	0.658457	0.658457
6	1	0.648666	-0.351334
7	0	0.729475	0.729475
8	1	0.702969	-0.297031
9	1	0.713664	-0.286336
10	1	0.644916	-0.355084
11	1	0.739848	-0.260152
12	1	0.726101	-0.273899
13	0	0.664091	0.664091

And this continues...

Each new tree focuses on what's still wrong. Each step makes the model slightly better. Not by being perfect—But by being **less wrong than before**.

Assuming we are training Tree 2:

```
df['Predict Residual_r1'] = np.random.rand(len(df))
df['Predicted y_2'] = df['Predicted y_1'] + learning_rate *
df['Predict Residual_r1']
print(df[['Predicted y_1', 'Residual r_1', 'Predict Residua
l_r1', 'Predicted y_2']])
```

	Predicted y_1	Residual r_1	Predict Residual_r1	Predicted y_2
0	0.680311	0.680311	0.592415	0.739553
1	0.737929	0.737929	0.046450	0.742574
2	0.716057	-0.283943	0.607545	0.776811
3	0.702723	-0.297277	0.170524	0.719775
4	0.658459	-0.341541	0.065052	0.664964
5	0.658457	0.658457	0.948886	0.753345
6	0.648666	-0.351334	0.965632	0.745229
7	0.729475	0.729475	0.808397	0.810314
8	0.702969	-0.297031	0.304614	0.733430
9	0.713664	-0.286336	0.097672	0.723432
10	0.644916	-0.355084	0.684233	0.713339
11	0.739848	-0.260152	0.440152	0.783863
12	0.726101	-0.273899	0.122038	0.738305
13	0.664091	0.664091	0.495177	0.713609

The following code randomly generated the predicted residuals for teaching purpose only. The real algorithm actually train regression trees. These trees l**earn patterns in residuals and r**educe error step by step.

When Do We Stop?

We stop when we reach a predefined number of trees.

- Maybe **10 trees**
- Maybe **100 trees**
- Maybe more, depending on the problem

Gradient Boosting is not about a single perfect model. It's about **a sequence of imperfect learners**, each correcting the last.

What I love about this algorithm is how *human* it feels. We don't demand perfection from the start. We allow mistakes. We measure them. And then we **learn from them—step by step**. That's Gradient Boosting.

- **Step 5: Making decision**

Now we've reached the part where many people pause and ask:

> *"Wait... everything is continuous now... so how do we actually make a final decision?"*

And you're absolutely right. Even though this is a **classification problem**, Gradient Boosting has been working entirely in **non-binary values** up to this point. **From Scores → Probabilities → Decisions**

At the end of Tree 2, what we have is: $\hat{y}_2$. These are **scores**, not final class labels. They represent the model's confidence—but not yet in a clean probability form.

This is where our hero steps in: Sigmoid Function

The Sigmoid function transforms any real number into a value between **0 and 1**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

So we take every value in `Predicted y_2` and pass it through Sigmoid:

$$p_i = \sigma(\hat{y}_2)$$

Now we finally have something meaningful:

- Values close to 1 → **likely "Yes"**
- Values close to 0 → **likely "No"**

But How Do We Decide the Class?

Normally, we would use a fixed threshold like:

$$p_i \geq 0.5 \Rightarrow \text{Yes}, \quad \text{else No}$$

But here, for teaching purposes, we'll do something more intuitive: **Using the Mean as a Threshold.** Instead of 0.5, we compute: $\text{threshold} = \text{mean}(\hat{y}_2)$

This gives us a **data-driven boundary** based on the model's current predictions.

Then:

- If $\hat{y}_2 \geq \text{mean} \rightarrow$ **Predict "**Yes**"**
- If $\hat{y}_2 < \text{mean} \rightarrow$ **Predict "**No**"**

Why This Works (Conceptually)? Think about what Gradient Boosting has been doing:

- It started with a rough guess
- Then kept adjusting predictions using residuals
- Now, each value in $\hat{y}_2$ reflects how strongly the model leans toward a class

By using the **mean**, we're essentially saying:

> *"Let's split the predictions into above-average confidence and below-average confidence."*

It's not standard practice—but it's a great way to **see the separation happening**.

Gradient Boosting doesn't directly output "Yes" or "No". It builds a **scoring system first**, then converts it into decisions using functions like Sigmoid and thresholds.

That's why it feels different from something like a single Decision Tree. It doesn't jump to conclusions. It **builds confidence first and decides later.**

```python
# Use the sigmoid function to convert the predicted y_2 to
probability
def sigmoid(x):
  return 1 / (1 + np.exp(-x))

df['Predicted Probability y_2'] = sigmoid(df['Predicted y_
2'])
df['Predicted Buy_XBOX'] = np.where(df['Predicted Probabili
ty y_2'] >= np.mean(df['Predicted Probability y_2']), 1, 0)
df['Actual Buy_XBOX'] = df['Buy_XBOX']
print(df[['Predicted Probability y_2', 'Predicted Buy_XBO
X', 'Actual Buy_XBOX']])
# Accuracy
print(f"Accuracy: {np.mean(df['Predicted Buy_XBOX'] == df
['Actual Buy_XBOX'])}")
```

	Predicted Probability y_2	Predicted Buy_XBOX	Actual Buy_XBOX
0	0.667854	0	0
1	0.680543	1	0
2	0.678411	1	1
3	0.680310	1	1
4	0.668554	0	1
5	0.665428	0	0
6	0.670368	0	1
7	0.677744	1	0
8	0.675284	1	1
9	0.679244	1	1
10	0.666083	0	1
11	0.693889	1	1
12	0.678322	1	1
13	0.671619	0	0

Accuracy: 0.6428571428571429

We've reached the moment of truth. We took our predictions, passed them through the **Sigmoid Function**, applied a threshold, and finally got binary outputs.

And the result? `Accuracy: 0.64` → Not impressive. But here's the thing, **that doesn't matter.** Because this section was never about performance. It was about understanding the **process**.

Gradient Boosting is not a model you judge after two trees. It's a model that **evolves**—slowly, carefully, and deliberately.

3. Gradient Boosting with Python

Here's how you would build a Gradient Boosting model in Python using the Scikit-Learn library:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import accuracy_score

# Load the dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name='target')

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, t
est_size=0.2,
                                                    random_
state=42)


# Train the model
# n_estimators is basically the number of decision trees (1
00 is default)
model = GradientBoostingClassifier(n_estimators=100, random
_state=42)
model.fit(X_train, y_train)

# Optional: Evaluate
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.956140350877193
```

```
# Feature Importance
importances = model.feature_importances_
feature_names = X.columns
feature_importance_df = pd.DataFrame({'Feature': feature_na
mes, 'Importance': importances})
print(feature_importance_df.sort_values(by='Importance', as
cending=False))
```

Feature	Importance
mean concave points	0.450528
worst concave points	0.240103
worst radius	0.075589
worst perimeter	0.051408
worst texture	0.039886
worst area	0.038245
mean texture	0.027805
worst concavity	0.018725
concavity error	0.013068
area error	0.008415
radius error	0.006870
worst smoothness	0.004811
fractal dimension error	0.004224
texture error	0.003604
mean compactness	0.002996
compactness error	0.002511
mean smoothness	0.002467
concave points error	0.002038
worst symmetry	0.001478
perimeter error	0.001157
mean concavity	0.000922
symmetry error	0.000703
smoothness error	0.000556
mean symmetry	0.000520
worst compactness	0.000450
mean area	0.000425
mean perimeter	0.000201
worst fractal dimension	0.000175
mean fractal dimension	0.000107
mean radius	0.000013

3. Beyond Vanilla Gradient Boosting

Now, once you understand the core idea, the natural question is: *"Can we make it better?"*

And that's exactly where these three variants come in.

- **XGBoost**
  XGBoost takes Gradient Boosting and makes it more disciplined. It introduces **regularization (L1 and L2)** to control model complexity and prevent overfitting. It also typically uses a **much smaller learning rate**, meaning it learns more slowly—but more precisely. Think of it as a more cautious learner that double-checks every correction before committing to it.
- **LightGBM**

  LightGBM changes how trees grow. Instead of expanding level by level, it grows **leaf-wise**, focusing only on the parts of the data with the **largest residuals**. In other words, it spends most of its effort where the model is most wrong. This makes it extremely fast and efficient, especially on large datasets—but also a bit more aggressive.
- **CatBoost**

  CatBoost is built with a very specific goal: handling **categorical variables** without manual encoding. It removes the need for preprocessing tricks like one-hot encoding and instead learns directly from raw categorical data. It also uses **symmetric trees**, where all leaves are at the same depth, giving it a more stable and structured learning process.

Summary

*Gradient Boosting is another ensemble model with multiple learners, but unlike methods where models work independently, here they **depend on each other**.*

Each learner is trained **sequentially**, correcting the mistakes of the previous one. Instead of trying to get everything right at once, the model improves step by step—turning small, weak learners into a strong predictor over time.

It doesn't rush to a decision. It builds understanding first. And only then does it decide.

References

https://www.geeksforgeeks.org/machine-learning/ml-gradient-boosting/

https://www.ibm.com/think/topics/gradient-boosting

https://developers.google.com/machine-learning/decision-forests/intro-to-gbdt

https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

https://www.youtube.com/watch?v=3CC4N4z3GJc

Next Section:

🤖 12. Adaptive Boosting